

# Bitcoin, Cryptocurrencies, and Blockchain technology

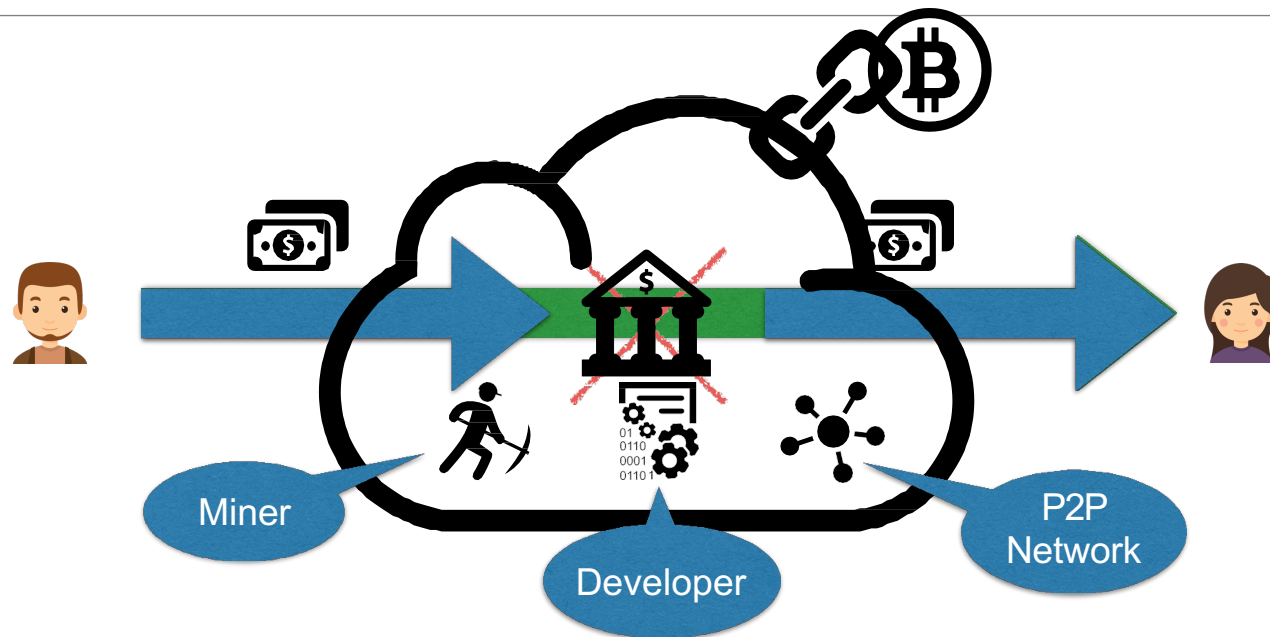
1. Bitcoin & Cryptocurrencies
2. Blockchain Technology
3. Security and Privacy in  
Bitcoin and Cryptocurrencies

# Outline

---

1. Bitcoin & Cryptocurrencies
  - Decentralised Payment System: A Big Picture
  - Cryptography in Bitcoin
  - Bitcoin Protocol: an Overview
2. Bitcoin Protocol and Blockchain Technology
  - Bitcoin Addresses
  - Bitcoin Distributed Ledger: a Blockchain
  - Transactions in Bitcoin and Blockchain
  - Consensus Mechanism: Proof-of-Work
3. Security and Privacy in Bitcoin and Blockchain

# Back to Bitcoin



1. How to perform secure **decentralized** payments in Bitcoin?
2. How to exchange **privacy-preserving** payments?
3. How to make decentralized systems **efficient**?



# Bitcoin and Blockchain Technology

---

# Blockchain Technology

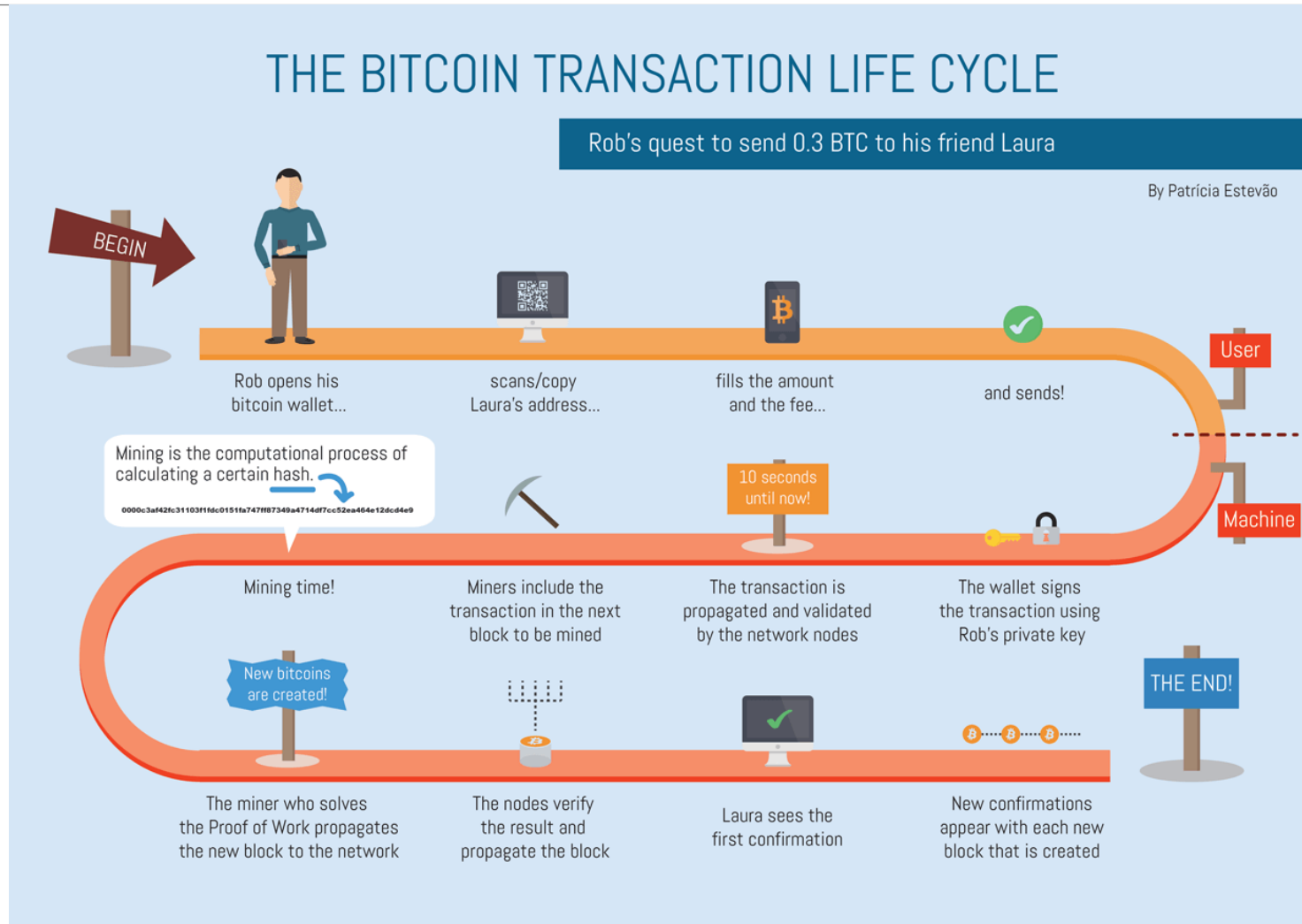
---

- Blockchain is the technology behind Bitcoin
  - Understanding How Bitcoin protocol is to understand Blockchain
- Bitcoin underpins Blockchain tech, but Blockchain goes further than only Bitcoin



<https://bitsonblocks.net/2015/09/09/a-gentle-introduction-to-blockchain-technology/>

# Bitcoin Transaction Lifecycle



<https://www.bambora.com/en/ca/blog/bitcoin-explained/>

# Bitcoin Transaction Lifecycle

---

1. Bob opens his bitcoin wallet.
2. Bob wants to transfer bitcoins to Alice. He scans Laura's bitcoin address.
3. Bob fills an amount of bitcoin he wants to transfer and the fee he is willing to pay AND form a transaction.
4. Before sending the new transaction to the blockchain, it is signed using Bob's private key signs it.
5. Now, the transaction is sent to the closest node on the bitcoin network and propagated into the network and verified.

# Bitcoin Transaction Lifecycle

---

6. It's mining time and miners pick up the transactions and group them into blocks trying to solve the Proof-Of-Work.
7. The miner who get it propagates the new block to the network.
8. The nodes verify the result and propagate the block.
9. Now Laura sees the first confirmation.
10. 10. New confirmations appear with each new block that is created and linked.



# Bitcoin Protocol

---

1. Bitcoin Addresses
2. Bitcoin Distributed Ledger: Bitcoin Blockchain
3. Bitcoin Transactions
4. Consensus Mechanism: Proof-of-Work

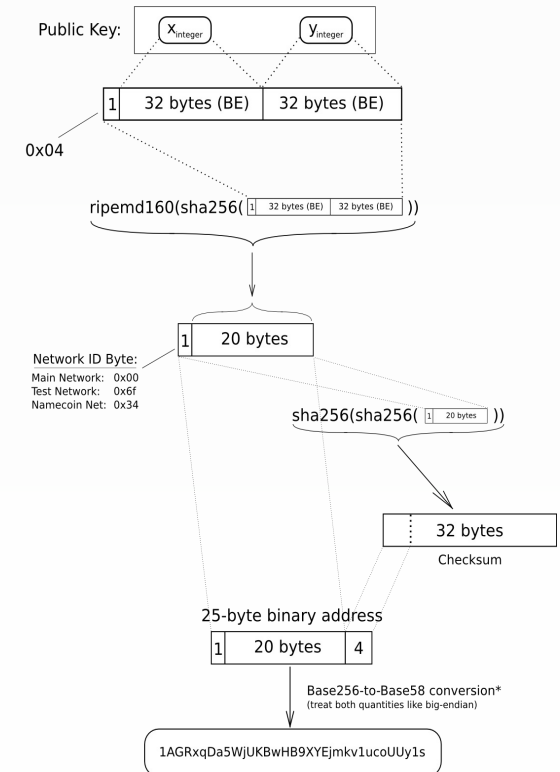
# Bitcoin Address

- Each user owns a key pair  $(pk, sk)$
- BTC address is generated from user's public-key:
  - Unique identifier
  - Hash of hash of a public key

F.Y.I: Total Balance < 21M BTC

Satoshi ==  $10^{-8}$  Bitcoin

## Elliptic-Curve Public Key to BTC Address conversion



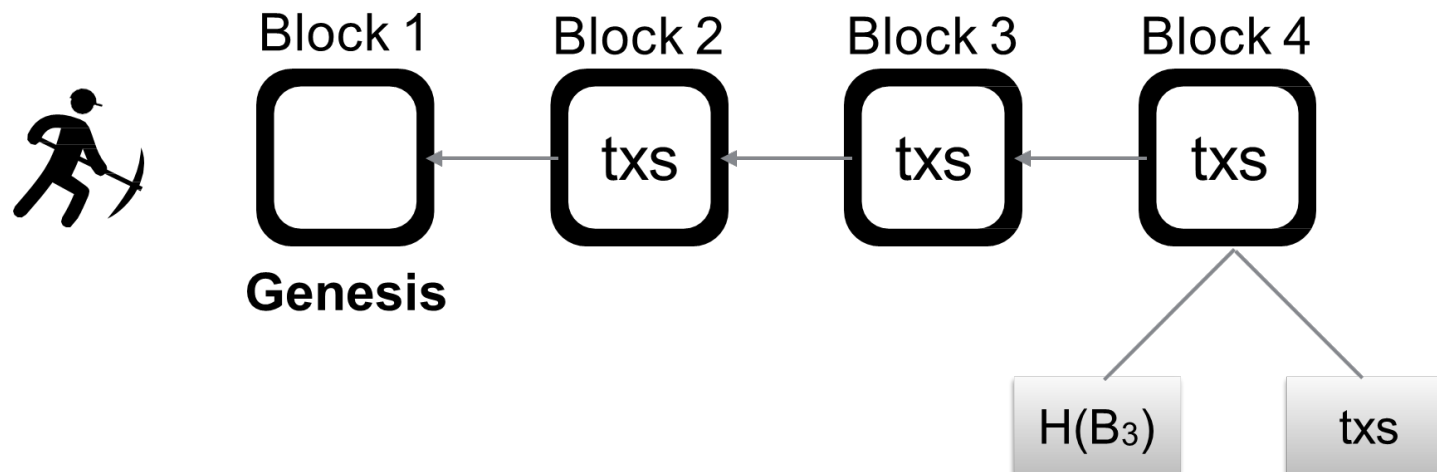
\*In a standard base conversion, the 0x00 byte on the left would be irrelevant (like writing '052' instead of just '52'), but in the BTC network the left-most zero chars are carried through the conversion. So for every 0x00 byte on the left end of the binary address, we will attach one '1' character to the Base58 address. This is why main-network addresses all start with '1'.

etotheipi@gmail.com / 1Gffm7LKXcNFPrtxy6yF4JB0e5rVka4sn1

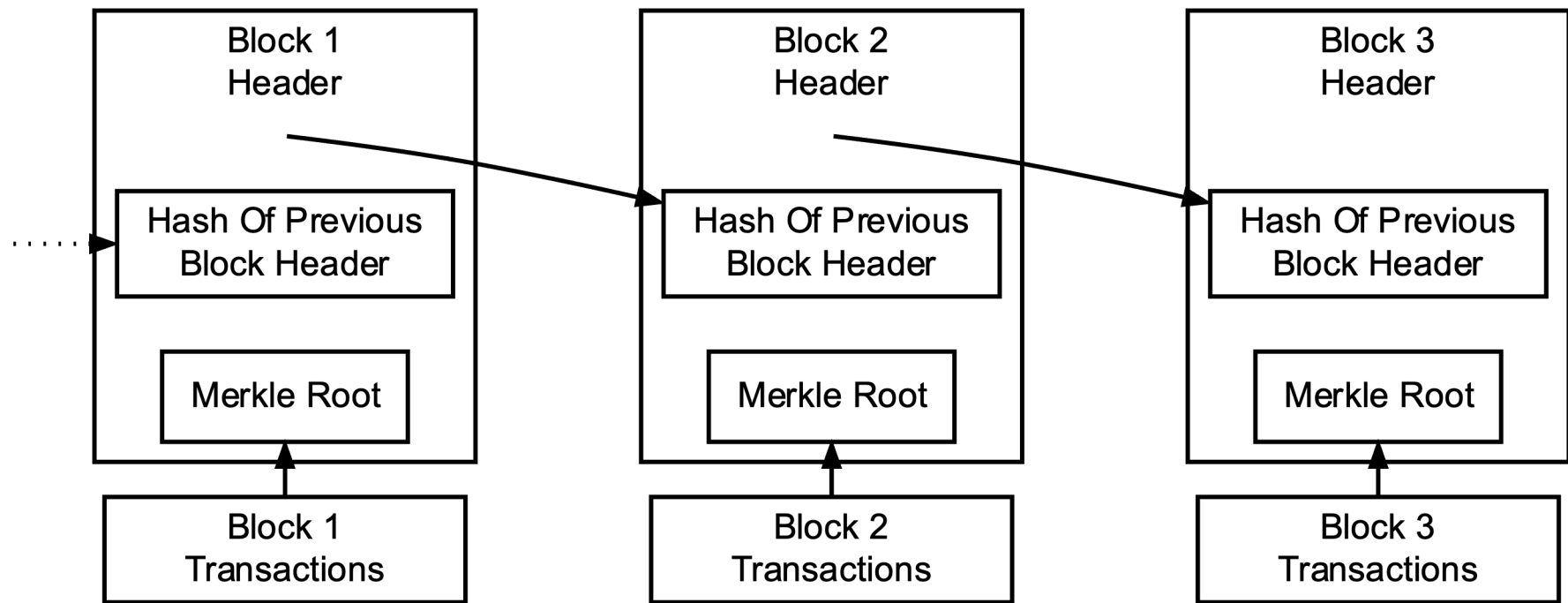
*1EGam2BeXd8sgphB44mEYqnDyDszw4YTER*  
*3J98t1WpEZ73CNmQviecrnyiWrnqRhWNLy*

# Bitcoin Ledger: a Blockchain

- Bitcoin Ledger is a chain of blocks (i.e., Bitcoin blockchain)
- Each block contains **Header** and **Transactions**
- Nodes in the Bitcoin network are expected to store exactly the same ledger → reach the consensus



# Bitcoin Ledger: a Blockchain

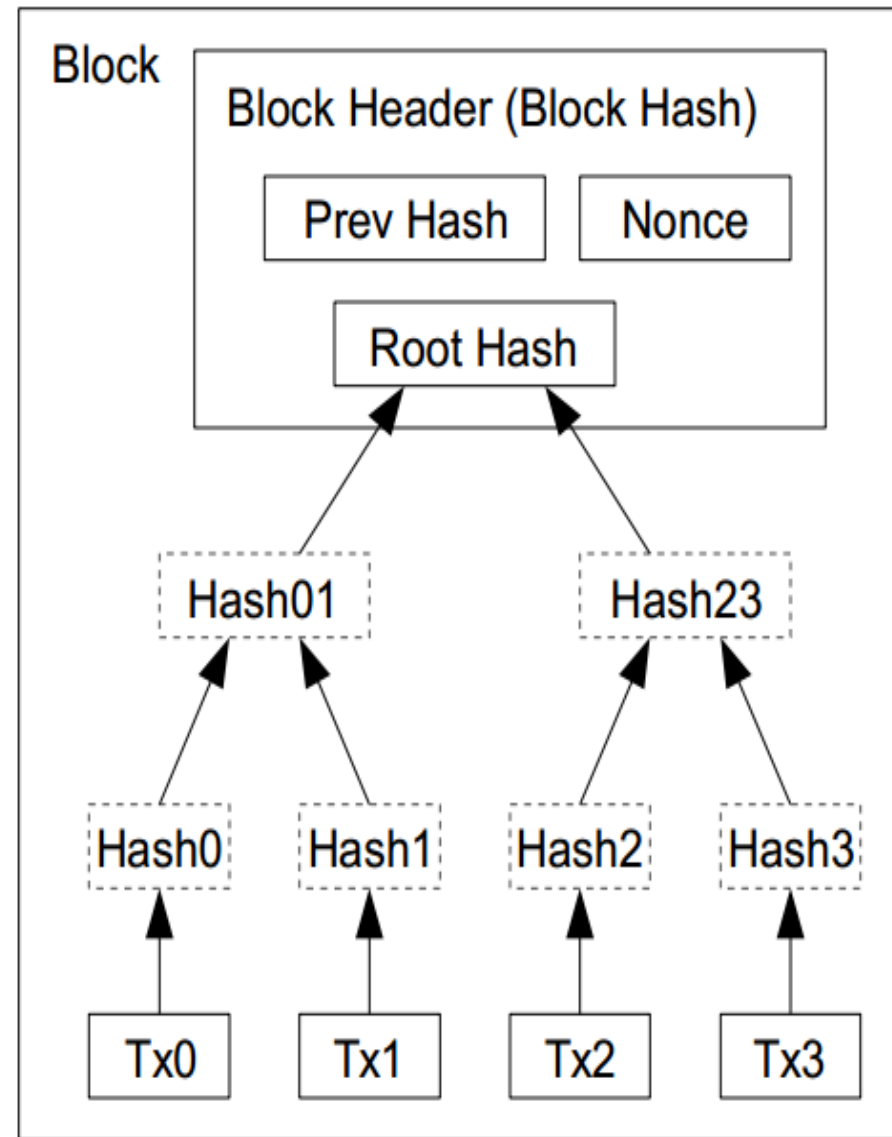


Simplified Bitcoin Block Chain

[https://developer.bitcoin.org/devguide/block\\_chain.html](https://developer.bitcoin.org/devguide/block_chain.html)

# Bitcoin Transaction

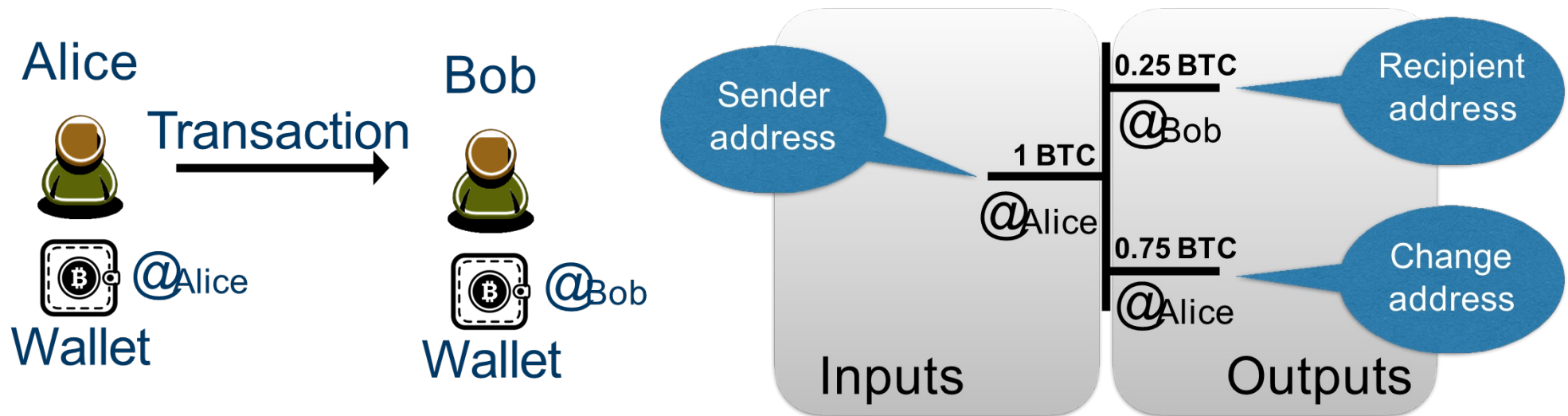
- Data Types: **Merkle Tree**
  - Root hash of the Merkle Tree is written in the header
  - If any transaction is modified, then the Root hash is changed → resulting in the change of the hash of Block header.
- Hash Function SHA-256 is used



Transactions Hashed in a Merkle Tree

# Bitcoin Transaction

- Transaction: Alice transfers 0.25 BTC to Bob



# Bitcoin Transaction

---

- Transaction Data Type: “Input”, “Output” and other parameters
- Input of the transaction is an output of another transaction.
  - Alice needs to claim she has 1BTC by showing the output of another transaction (which is 1BTC) belong to her.
  - To do that, Alice needs to use her private-key to generate her digital signature
- Outputs: Define conditions using a scripting system
  - The conditions must be satisfied in order to spend the output in the next transactions.
  - Digital signature is used to sign the transaction, and Elliptic Curve Digital Signature Algorithm (*secp256k1*) is used

1. {"hash":"7c4025...",      *Content of a Bitcoin transaction*  
2. "ver":1,                      *with 1 input and 2 outputs.*  
3. "vin\_sz":1,                  *Transaction verification using a*  
4. "vout\_sz":1,                  *scripting system*
- 
5. "lock\_time":0,  
6. "size":224,  
7. "in":[      ← **Tx Input**  
8. {"prev\_out":  
9. {"hash":"2007ae...",  
10. "n":0},  
11. "scriptSig":"304502... 042b2d..."}],  
12. "out":[      ← **Tx Output**  
13. {"value":"0.25",  
14. "scriptPubKey":"**OP\_DUP OP\_HASH160 a7db6f**  
**OP\_EQUALVERIFY OP\_CHECKSIG"**} ← **Condition to**  
    {"value":"0.75",                      **Bob**  
15. "scriptPubKey":"**OP\_DUP OP\_HASH160 34sa6f**  
**OP\_EQUALVERIFY OP\_CHECKSIG"**}} ← **Condition to**  
                                                 **Alice**



# Bitcoin Scripting System

---

- Stack-based programming language
- If evals to true → Bitcoin transaction is valid
- Many opcodes defined
- Execution time is critical to prevent DoS attacks

## Example Script

<https://developer.bitcoin.org/reference/transactions.html>

`<signature><publicKey>`

Constants  
are pushed onto  
the stack

`OP_CHECKSIG`

Operation  
executed on  
stack values

# Consensus Mechanism: Proof of Work

---

- Bitcoin is a “distributed ledger blockchain”, therefore:
  - Information added to the ledger must be accurate and honest
  - Entire network agrees with the ledger’s content
- A mechanism for all nodes in Bitcoin network to cooperate and reach to a common opinion (consensus):
  - Reach agreement on adding blocks to the blockchain
  - Keep all nodes in the network synchronized

# Consensus Mechanism: Proof of Work

---

- Bitcoin requires that each block prove a significant amount of work was invested in its creation.
  - This ensures that untrustworthy peers who want to modify past blocks have to work harder than honest peers who only want to add new blocks to the block chain.
- Chaining blocks together makes it impossible to modify transactions included in any block without modifying all subsequent blocks.
  - The cost to modify a particular block increases with every new block added to the block chain, magnifying the effect of the proof of work.

# Consensus Mechanism: Proof of Work

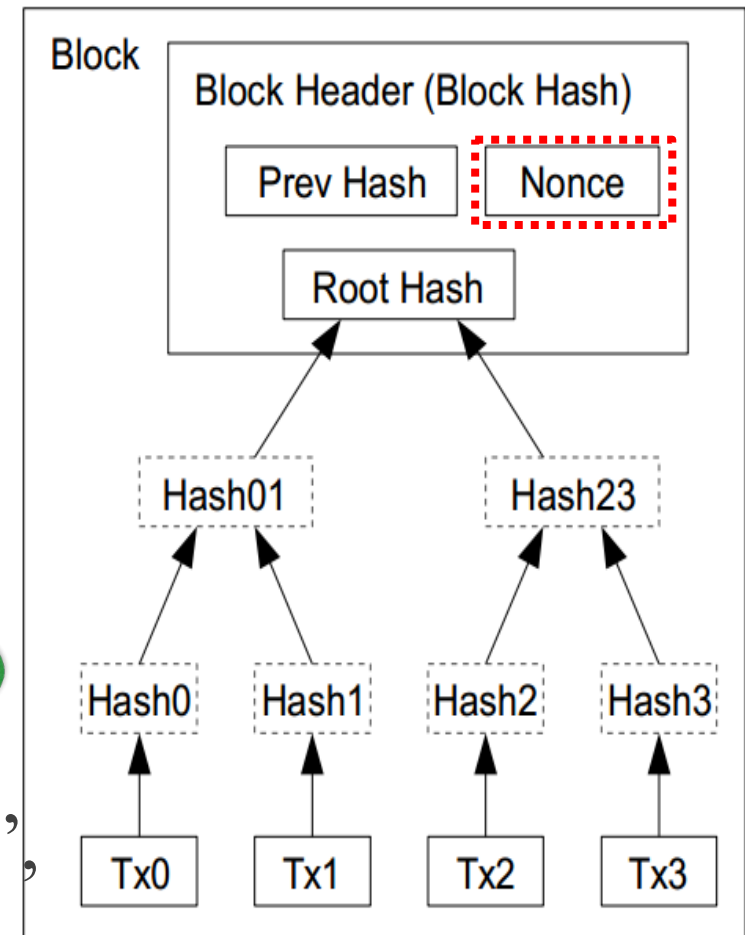
- Nonce?
  - Number only used once
- Some nodes in the Bitcoin network try to find Nonce N, s.t.:

$$\text{Hash}(\text{Hash}(B_3)|\text{txs}|N) < \text{target}$$

Best known approach: **Brute Force**

Controls the **difficulty**

- This process is known as “mining”, and the nodes are “miners”



Transactions Hashed in a Merkle Tree

# Proof of Work: Example

**Hash (Hash (B<sub>3</sub>) | txs | N) < target = 0x000\*\***

---

Hash(Block\_3 | merkle\_root | 0xbeed) = 0x03ef..

Hash(Block\_3 | merkle\_root | 0xbeee) = 0x12ef..

Hash(Block\_3 | merkle\_root | 0xbeef) = 0x000f.. ✓

*This miner successfully find the Nonce, as a result correctly form a new block. He claims his work by broadcasting this block to the network*

**The verification is easy. But Proof-of-Work is hard.**

# Proof of Work: Example

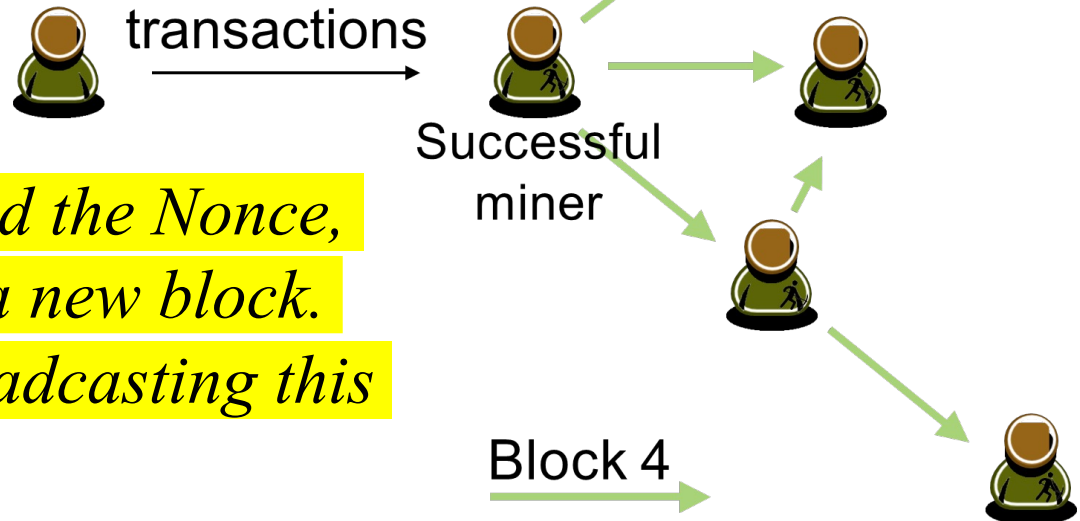
**Hash (Hash (B<sub>3</sub>) | txs | N) < target = 0x000\*\***

---

Hash(Block\_3 | merkle\_root | 0xbeed) = 0x03ef..

Hash(Block\_3 | merkle\_root | 0xbeee) = 0x12ef..

Hash(Block\_3 | merkle\_root | 0xbeef) = 0x000f.. ✓



*This miner successfully find the Nonce,  
as a result correctly form a new block.  
He claims his work by broadcasting this  
block to the network*

# Bitcoin Protocol: Update Rules and Forks

---

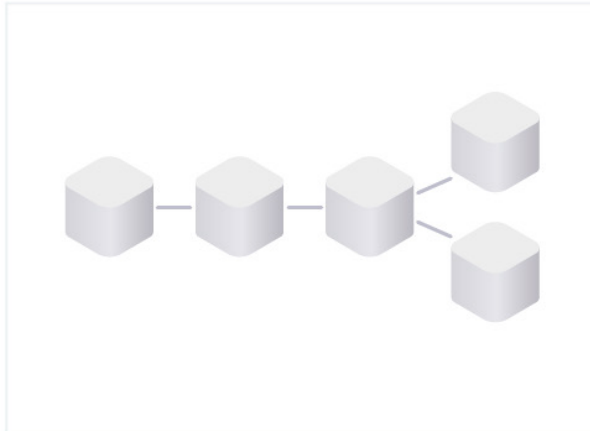
- What happens if **some miners** successfully calculate “Nonce” and broadcast their own new block to the network
  - Network partition
  - Fork
    - Soft-fork: temporary
    - Hard-fork: permanent
- Update Rule:
  - **Longest chain wins**



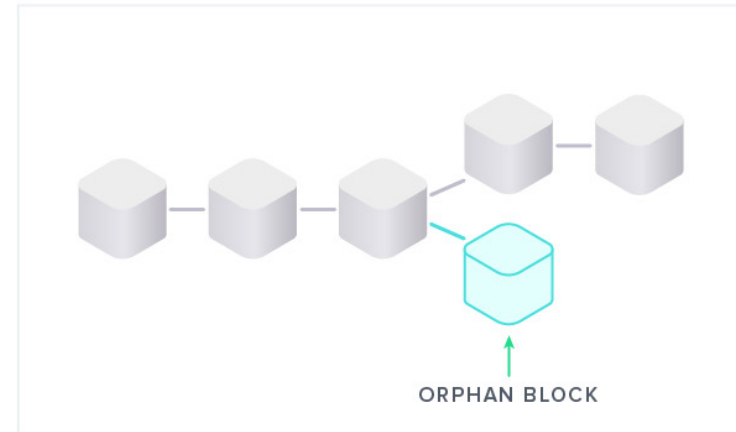


# LONGEST CHAIN RULE

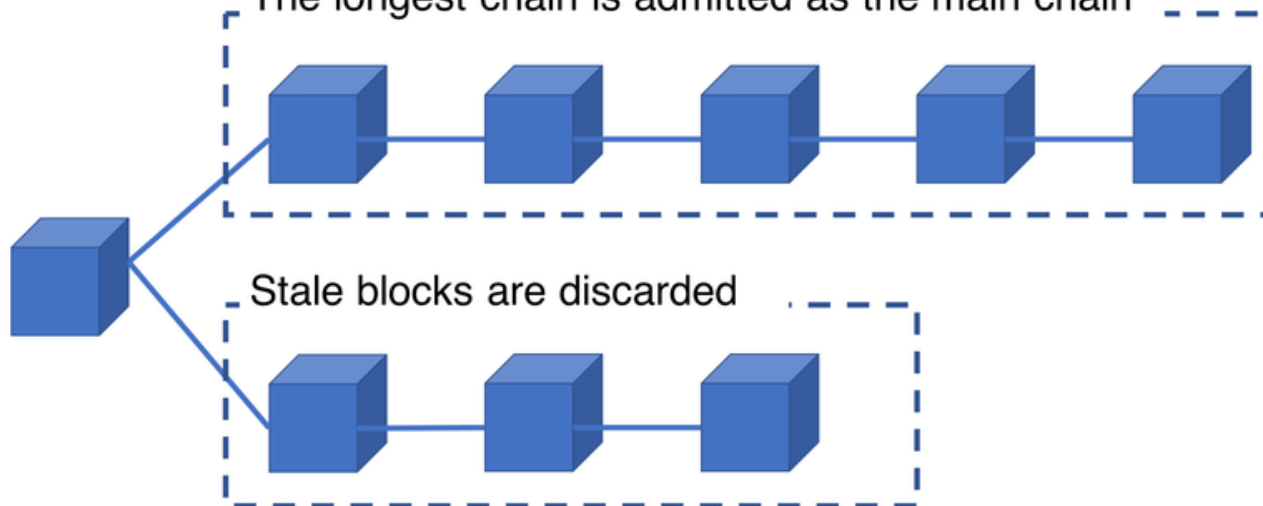
Two valid blocks found and broadcasted at the same time



The next block that is found breaks the tie



The longest chain is admitted as the main chain



*Eventually all the nodes will have a same blockchain (the longest one).*

# Summary

---

We have learned about the Bitcoin Protocol, which underpins the Blockchain technology

- Bitcoin Addresses
- Bitcoin Distributed Ledger: a chain of Blocks
- Transactions in Bitcoin and Blockchain
- Consensus Mechanism: Proof-of-Work

# Quiz

---

6. What cryptographic algorithms are used by bitcoin?
7. In a transaction 1-input 2-output, one output belongs to Bob, whose is another output?
8. What is distributed consensus and how is it achieved with bitcoin?
9. Why Proof-of-Work is hard but easy to verify?

# Practice

---

## Bitcoin key-pair generation:

- Private key:
- Public key:
- Address:
- Using OpenSSL

# Experiment with OpenSSL command line

---

- OpenSSL contains a large set of pre-defined curves that can be used. The full list of built-in curves can be obtained through the following command:

```
$ ecparam -list_curves
```

- An EC Parameters file contains all of the information necessary to define an Elliptic Curve that can then be used for cryptographic operations (for OpenSSL this means ECDH and ECDSA):

```
$ ecparam -name secp256k1 -out secp256k1.pem
```

# Experiment with OpenSSL command line

---

- Information on the parameters that have been used to generate the key are embedded in the key file itself.

```
$ ecparam -in secp256k1-key.pem -text -noout
```

- This will simply confirm the name of the curve in the parameters file. If you wish to examine the specific details of the parameters associated with a particular named curve then this can be achieved as follows:

```
$ ecparam -in secp256k1.pem -text -param_enc  
explicit -noout
```

# Experiment with OpenSSL command line

---

- Keys can be generated from the `ecparam` command, either through a pre-existing parameters file or directly by selecting the name of the curve. To generate a key pair from a pre-existing parameters file use the following:

```
$ ecparam -in secp256k1.pem -genkey -noout -out  
secp256k1-key.pem
```

- Or to do the equivalent operation without a parameters file use the following:

```
$ ecparam -name secp256k1 -genkey -noout -out secp256k1-  
key.pem
```

- To see the keys and the curve info:

```
$ ec -in secp256k1-key.pem -noout -text
```

# Experiment with OpenSSL command line

---

- We can also generate a private key as follows:

```
$ ecparam -name secp256k1 -genkey -noout -out  
priv-key.pem
```

- The public key can be obtained using this command:

```
$ ec -in priv-key.pem -pubout -out pub-  
key.pem
```